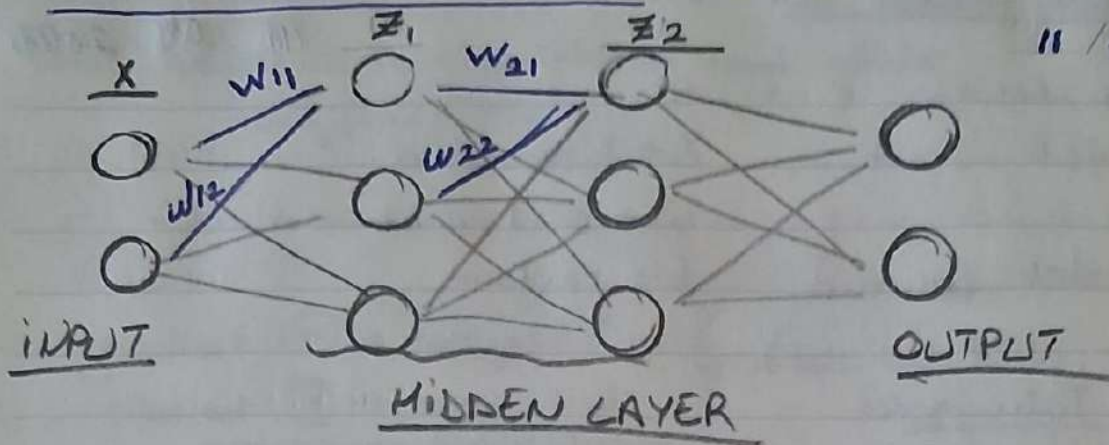


Artificial Neural Network (ANN)

11 / 09 / 26



1) Tek nöron için :

2) Aktivasyon fonksiyonu

Girdi $\frac{\text{weight}}{\text{input}} \times \text{bias}$

$$Z = Wx + b$$

$$y = F(Z)$$

3) Los değerinin hesaplanması

4) Backpropagation (geriye doğru gradient tırmanı) } Zincir kuralı

5) Training loop

X
↓
Linear

↓
Relu

↓
Linear

↓
Sigmoid

↓
Prediction

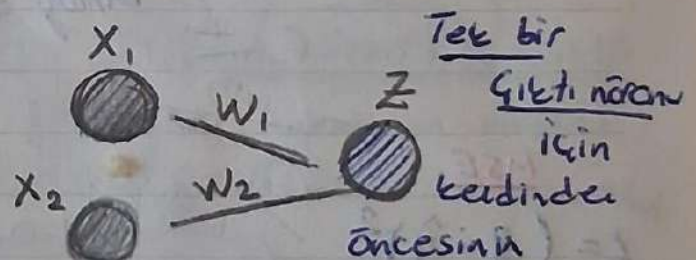
↓
Loss

weight update

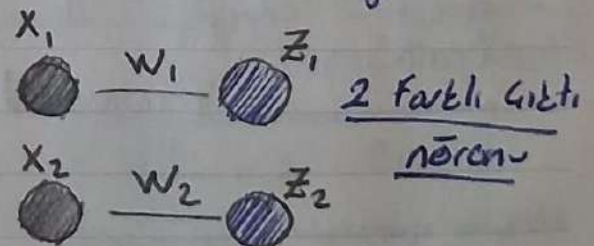
$$\frac{\partial L}{\partial w_1}$$

$$\frac{\partial L}{\partial w_2}$$

$$Z = (X_1 w_1) + (X_2 w_2) \dots + b$$



Girdi * Ağırlık + Bias



ANAG

x_1	x_2	Cevap
0	0	0
1	0	1
0	1	1
1	1	1

$$Z_1 = X_1 * w_1 + b$$

$$Z_2 = X_2 * w_2 + b$$

ADIM-1) Dış iskelet kur ✓

11 / 09 / 2026

Class Neuron

def __init__

X → kullanıcının girileceği değerler

W → random

def forward

b → random

ADIM-2) Tahmin

$$X \cdot W + b = Z$$

Sigmoid

input → weighted sum → ~~bias~~ → activation → prediction func

$$Z = X \cdot W + b$$

$$\hat{y} = f(Z) \rightarrow \text{Nöronun tahmini}$$

1. Aktivasyon Fonksiyonu $Z \rightarrow$ Sigmoid

$$\sigma(Z) = \frac{1}{1 + e^{-Z}}$$

$Z \rightarrow -\infty$ sigmoid = 0 → hayır

$Z \rightarrow 0$ sigmoid = 0,5

$Z \rightarrow +\infty$ sigmoid = 1 → evet

Binary classification probleminde
kullanacağız

ADIM-3) Loss Func

Tahmin ne kadar yaklaştı?

hata

$$L = (y - \hat{y})^2$$

MSE

gerçek cevap
↓
Tahmini cevap

ör

$$\begin{cases} (1 - 0.9)^2 = 0.01 \\ (1 - 0.1)^2 = 0.81 \end{cases}$$

hata

MSE
Mean Square Error

forward() = dot product → sigmoid

↓

Nokta çarpımı yap

Loss Func

Sigmoid ile 0-1 arası değer al

Hata değerini hesapla (Target senelerisin)

ör Target 1 için sigmoidin sonucu ne kadar yakın??

ADIM-4) Gradient

11 / 09 / 2026

Loss'u azaltmak için weightleri hangi yönde değiştireceğiz?

$\frac{\partial L}{\partial w} \rightarrow$ hatırlarımızın belirtilen ağırlığa göre tervini alır

gradient $\ominus \rightarrow$ weight $\uparrow$
gradient $\oplus \rightarrow$ weight $\downarrow$ } Loss azalması için

$$w_{\text{yeni}} = w_{\text{eski}} - \underset{\substack{\downarrow \\ \text{learning} \\ \text{rate}}}{\eta} \cdot \underset{\substack{\downarrow \\ \text{gradient}}}{\frac{\partial L}{\partial w}}$$

Nöron
sigmoid $z = x \cdot w + b$
loss $\hat{y} = \sigma(z)$
loss $L = (y - \hat{y})^2$
gradient $\frac{\partial L}{\partial w}$

ZİNCİR TREVİ

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

MSE Tervİ

$$L = (y - \hat{y})^2$$

$$\frac{\partial L}{\partial \hat{y}}$$

Sigmoid Tervİ

$$\hat{y} = \sigma(z)$$

$$\frac{1}{1 + e^{-z}}$$

$$-2(y - \hat{y})$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

z'nin Tervİ

$$z = x \cdot w + b$$

$$\frac{\partial z}{\partial w} \quad \boxed{x}$$

$$\hat{y}(1 - \hat{y})$$

Sigmoidin kendisini kullanarak Tervİ yazılır

$$\frac{\partial z}{\partial b} = 1 \quad \text{Bias}$$

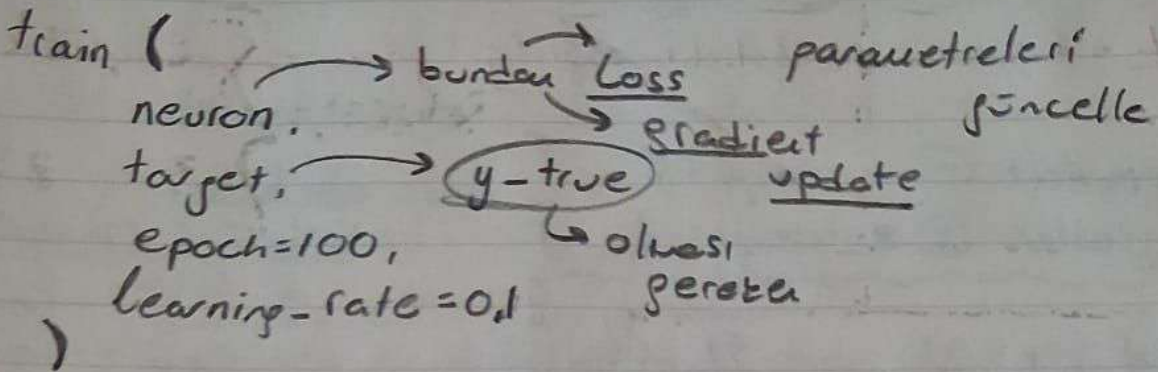
$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$\rightarrow [-2(y - \hat{y})] \cdot [\hat{y}(1 - \hat{y})] \cdot [x]$$

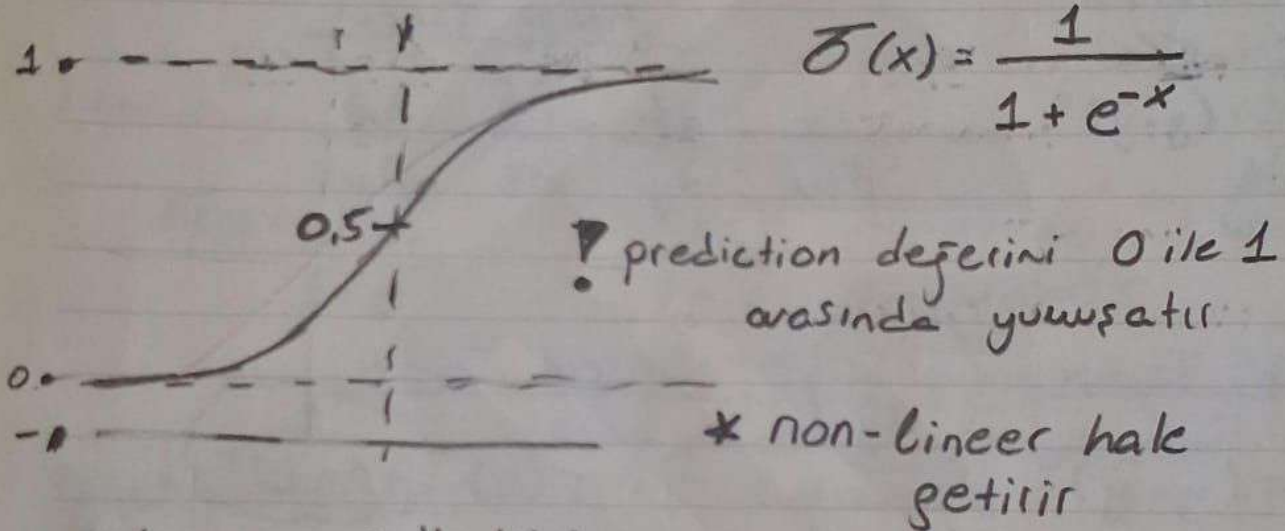
ADIM-5) Training

11 / 08 / 2026

Tahmin yap $\rightarrow$ Loss hesapla $\rightarrow$ Gradiert hesapla



Sigmoid Fonksiyonu



Her nöron belli bilgi

öğrenebilir sonraki katman bilgileri birleştirir.

z değeri $\uparrow\uparrow / \downarrow\downarrow \rightarrow$ Tarevi $\downarrow\downarrow$ gradiert küteler

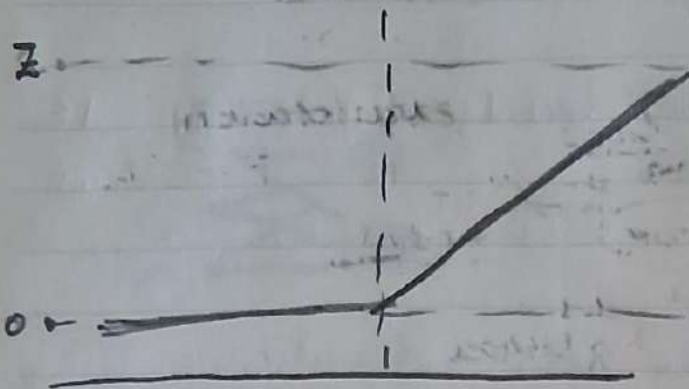
çok büyük $\rightarrow$ çok küçük

öğrene yavaşlar

(Rectified Linear Unit)

RELU Fonksiyonu

11/09/2016



$$\text{Relu}(z) = \begin{cases} 0, & z < 0 \\ z, & z \geq 0 \end{cases}$$

Tsrevi $\rightarrow$ $z < 0$ için Tanımsız $\emptyset$
 $z > 0$ için 1

* pozitif bölgede
ReLU gradienti
ölmez

Sigmoid - gradient

$z = -10 \rightarrow 0.000045$

$z = 0 \rightarrow 0.25$

$z = 10 \rightarrow 0.000045$

Relu

$z = -10$

$z = 0$

$z = 10$

gradient

0

0

1

$\rightarrow$ en büyük değeri bile
0.25

$\rightarrow$ gradienti konstante ve negatif
tarafta sifira gider

$\rightarrow$ Ağırlık değişimi çok $\downarrow$

ReLU negatifte

$$\frac{\partial L}{\partial w} = \dots \times \underbrace{\text{ReLU}'(z)}_{\emptyset} \times \dots$$

* ağırlık güncellenmez

Trevi sıfır olması sıkıntı yapar

Leaky Relu

12/09/2026

Normal Relu problem $\rightarrow$ Negatif taraf terev $\emptyset$

$$\frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w} = \emptyset \text{ gradıent deęiřimi}$$

--- $\emptyset$ ---

bu sorun özetek için Leaky Relu kullanılır.

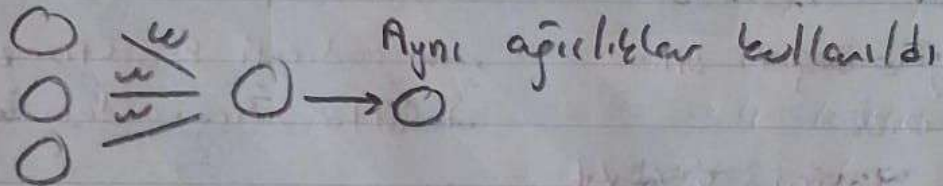
$$\text{Leaky Relu}(z) = \begin{cases} \alpha z, & z < 0 \\ z, & z \geq 0 \end{cases}$$

gradıent $\ominus$ de artık $\underline{\alpha}$

gradıent $\oplus$ de $\underline{1}$

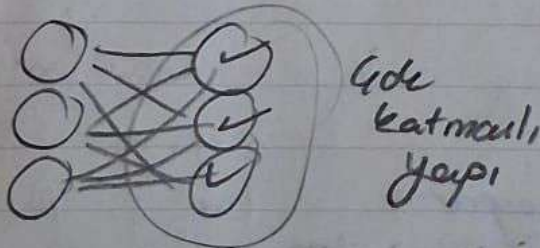
Yanlıř tahmin $\rightarrow$ Loss $\checkmark \rightarrow$ gradıent $\neq 0 \rightarrow$ weight deęiřimi $\checkmark$

řıca kadar yapılar her input her output



ok katmanlı sistem

- * Her nronun kendi ağırlığı vardır. hidden layer
- * +1 nronu yöneten yapıdır sistem



layer = { nron sayısı
input sayısı
aktivasyon fonk.
}

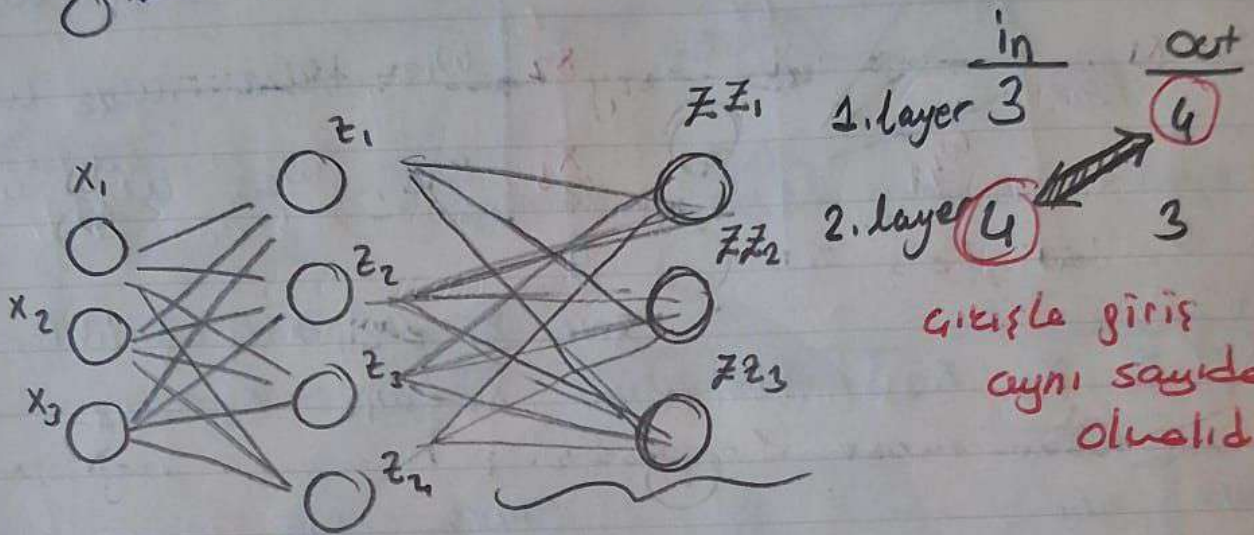
Layer Forward

12/09/2021

Layer Forward



Gitti iki elemanlı olmalı aynı
renkler kendi arasında
nereye neye yap-



girişle giriş
aynı sayıda
olmalıdır

girdimiz 4 input
çıkardık 3 output

çıkardık 4 tane
output

$$[zz_1, zz_2, zz_3]$$

$$[z_1, z_2, z_3, z_4]$$

3 girdi 4 çıktı

$$\begin{aligned} & * (z_1 \cdot ww_{11}) + (z_2 \cdot ww_{21}) \\ & + (z_3 \cdot ww_{31}) + (z_4 \cdot ww_{41}) + b \end{aligned}$$

$$\begin{aligned} & * (x_1 \cdot w_{11}) + (x_2 \cdot w_{21}) \\ & + (x_3 \cdot w_{31}) + b = z_1 \end{aligned}$$

2 Layer'i bağlamak için

3 input alır
4 output verir

layer1 = Layer(3, 4, "relu", "relu-derivative")

layer2 = Layer(4, 3, "relu", "relu-derivative")

→ sonuçta 3

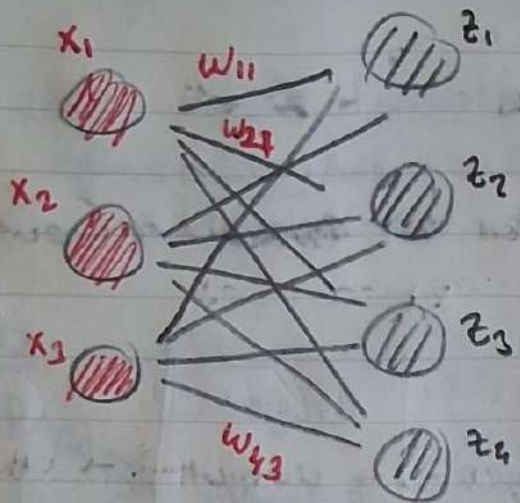
input = np.array([2.0, 1.0, 3.0])

3 input

output
çıkış
verir

Nöta Çarpım Matrisi

12/08/2026



Ağırlıklar için

NÖRONLAR

$$W = \begin{matrix} & \begin{matrix} z_1 & z_2 & z_3 & z_4 \end{matrix} \\ \begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix} & \begin{bmatrix} w_{11} & w_{21} & w_{31} & w_{41} \\ w_{12} & w_{22} & w_{32} & w_{42} \\ w_{13} & w_{23} & w_{33} & w_{43} \end{bmatrix} \end{matrix}$$

$$X = [x_1 \ x_2 \ x_3]$$

ya da z_1 etki eden ağırlıklar
 $x_1 \rightarrow w_{11} \quad x_2 \rightarrow w_{12} \quad x_3 \rightarrow w_{13}$

$$W = \begin{bmatrix} w_{11} \\ w_{12} \\ w_{13} \end{bmatrix}$$

ÖRNEK ÇEBİR

$$\begin{matrix} x \rightarrow 1,3 \\ w \rightarrow 2,4 \end{matrix} \left\{ \begin{matrix} \text{vektör} \\ \text{çarp} \end{matrix} \right. \rightarrow \boxed{1,4}$$

$$\begin{matrix} \text{scale} \\ b \rightarrow 4 \end{matrix} \rightarrow 4 \text{ elemanlı} \\ z \rightarrow 1,4 \quad \text{tek boyut}$$

$$(x_1 \cdot w_{11}) + (x_2 \cdot w_{12}) + \dots$$

✓ Nöta Çarpımı

$X \cdot W$ matris çarpımıdır.

⚠ bias değerleri sadece
sayı olarak toplanır

$$Z = X \cdot W + b$$

↳ weighted sum

Nöron için $\rightarrow$ nöta çarpım sonucu

önceki 3 input
için 3 sonuç

Son elimentleri liste

$$\text{Neuron-1} \rightarrow [1, 2, 3]$$

$$\text{Neuron-2} \rightarrow [4, 5, 6]$$

$$\text{input} \rightarrow [1, 2, 3]$$

$$\left. \begin{matrix} 3 \times 2 \\ 3 \times 1 \end{matrix} \right\} ?$$

Transpoz al
çarpmak
için

12/09/2026

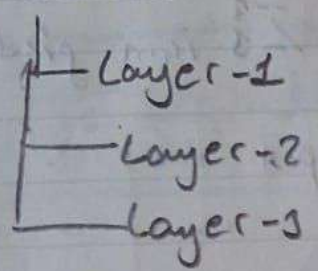
Neuron 1 Neuron 2

$$W = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \times \begin{matrix} x = \text{input} \\ [1, 2, 3] \end{matrix} = Z \checkmark$$

Ağırlıklar

önceki tür ki sonuçları (output) matris yap transpose al

Network



~~input~~ → Layer-1 → output-1 → Layer-2

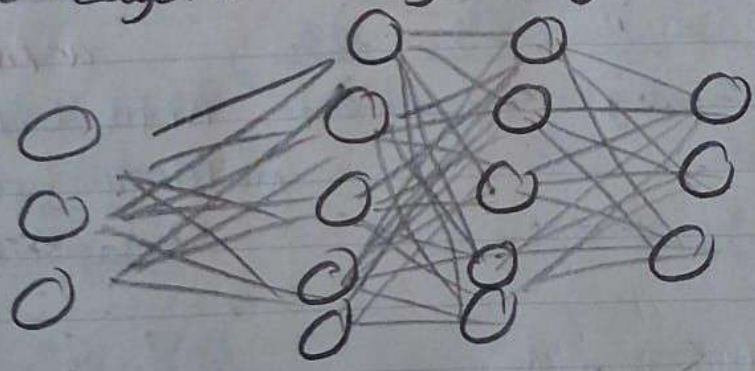
prediction ← Layer-3 ← output-2

Görev Ağırlımı

Nöron → kendi parametreleri (ağırlık / girdi / çıktı)

Layer → Nöronların hepsi / ağırlık / bias / W matrisi / b matrisi

Network → Layerleri sıraya koyar / Forward akışını yönetir

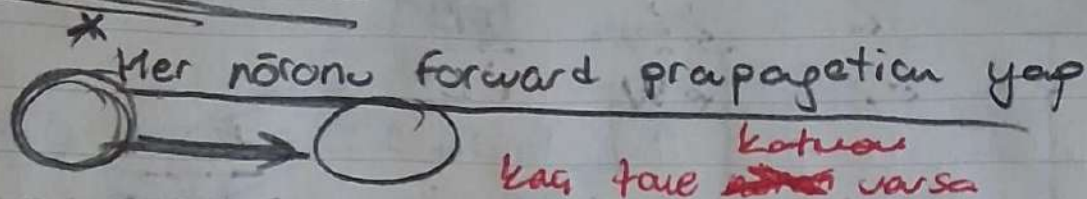


Akışı düzenler → Network içinde döngüyle

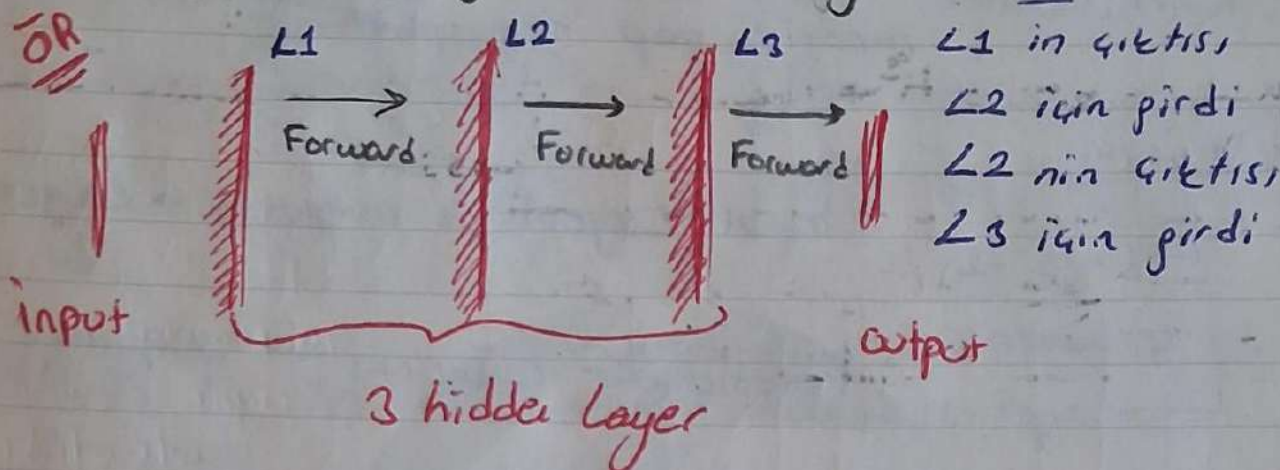
$3 \times \underline{5} \quad \underline{5 \times 5} \quad \underline{5 \times 3}$ yapacağız

Network Forward

12 / 09 / 2026



→ For layers in Self. Layers Note



Katmanlar içinde her katmanı layer forward (input) uygula

- Her 2 layer arasında Forward uygulasın
- Kaç layer olduğunda for döngüsü ile dönsün

Network Backpropagation

bizim elimizde W matrisi var
artık

Normalde → $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$

tek ağırlık için

Artık bütün W matrisinin gradienti hesaplanacak

$$Z = X \cdot W + b$$

$$A = f(Z)$$

Layerdeki nöronların aktivasyon fonk uygulanmış halidir

$$Y_{\text{aui}} = Z = [-2, 3, -1, 5]$$

↓ ReLU

$$A = [0, 3, 0, 5]$$

Sonraki
layerin
inputu

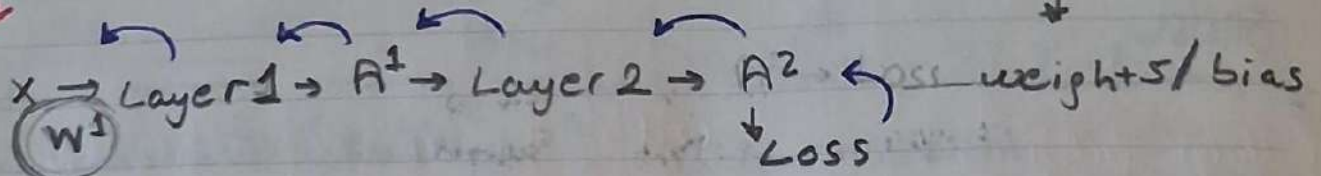
Backprop

12 / 09 / 2026

$$\frac{\partial L}{\partial A} \quad \frac{\partial L}{\partial Z} \quad \frac{\partial L}{\partial W} \quad \frac{\partial L}{\partial b}$$

Loss $\rightarrow$ output layer $\rightarrow$ önceri layer $\rightarrow$ önceri layer
 gradient gradient gradient

Yol



$$\frac{\partial L}{\partial W^1} = \frac{\partial L}{\partial A^2} \cdot \frac{\partial A^2}{\partial Z^2} \cdot \frac{\partial Z^2}{\partial A^1} \cdot \frac{\partial A^1}{\partial Z^1} \cdot \frac{\partial Z^1}{\partial W^1}$$

Backpropagation Temelde Nasıl Çalışır Örnek Üzerinde

$$X=2$$

$$W=3$$

$$b=1$$

$$Z = X \cdot W + b$$

$$2 \cdot 3 + 1$$

$$Z=7$$

$$A = \text{Relu}(7)$$

$$A=7$$

$$\text{Target} = 10$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial A} \cdot \frac{\partial A}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

$$\frac{\partial L}{\partial A} = -2(y - A) = -2(10 - 7) = -6$$

DENGELEME

MSE $L = (y - A)^2$
 $(10 - 7)^2$

$$L=9$$

$$b_{\text{new}} = b - \eta \frac{\partial L}{\partial b}$$

$$1 - 0.1(-6)$$

$$b_{\text{new}} = 1.6$$

$$\frac{\partial A}{\partial Z} = \text{Relu}'(Z)$$

$$\text{Relu}'(x) = 1$$

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial A} \cdot \frac{\partial A}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

$$\frac{\partial Z}{\partial W} = X$$

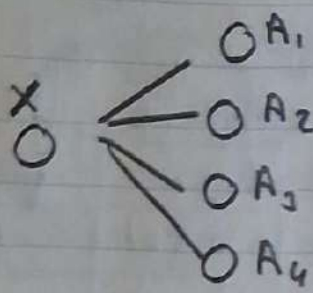
$$X=2 \rightarrow 2$$

$$(-6) \times (1) \times (2) = -12$$

$$W_{\text{new}} = W_{\text{old}} - \eta \cdot \frac{\partial L}{\partial W} = 3 - 0.1(-12) = 4.2$$

Gök ağır konu ! Sindiremedim

4 nöron varsa: $A = [A_1, A_2, A_3, A_4]$ 12/09/2026



$$\frac{\partial L}{\partial A} = \left[\frac{\partial L}{\partial A_1}, \frac{\partial L}{\partial A_2}, \frac{\partial L}{\partial A_3}, \frac{\partial L}{\partial A_4} \right]$$

ÖRNEK

Layer $\rightarrow$ Nokta çarpımı
Forward Aktivasyon fon } uygulanan nöronlardır

Çıktısı
(output)

Layer $\rightarrow$ Nöronlar $\rightarrow$ input
Layer, Forward $(X) = A$
 $* A = F(W \cdot X + b)$

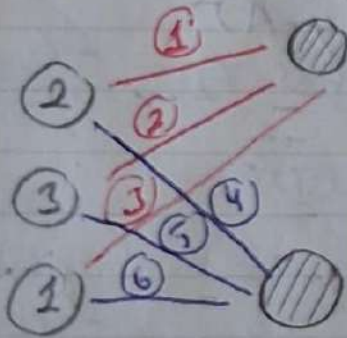
(input)

$X = [2, 3, 1]$ olsun

bias = $[1, 2]$

Ağırlıklar 2 nöron için

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \rightarrow \begin{matrix} 1. \text{ nöron} \\ 2. \text{ nöron} \end{matrix}$$



Nokta çarpımı

$$\begin{aligned} 2 \cdot 1 + 3 \cdot 2 + 1 \cdot 3 + b_1 &= 12 \\ 2 \cdot 4 + 3 \cdot 5 + 1 \cdot 6 + b_2 &= 31 \end{aligned}$$

Bunu mat için kısa yolda
matris çarpımı yap

3 input var ağırlıklar için de
 1×3 pirdi boyutu
3 olmalı

* TRANSPOZ AL

$$[2, 3, 1] \cdot \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} + [1, 2]$$

$$= X \cdot M + b$$

$$Z = [12, 31]$$

Nokta Çarpımı ✓
→ Aktivasyon Fonk

$$\text{ReLU}(z) \\ \downarrow \\ \text{ReLU}[12, 31]$$

13 / 09 / 2026

Layerin Forward A = [12, 31]
Çıktısı

Y

Loss
Fonksiyonu

Hedef değerimiz [10, 20] olsun

$$\text{Loss} = (Y-A)^2 + (Y-A)^2 \xrightarrow{\text{Trev}} (Y-A)^2 \rightarrow -2(Y-A)$$

matrisler
Fark

$$(10-12)^2 + (10-31)^2 = 125 \text{ bu kadar}$$

$$-2 \cdot [-2, -11]$$

$$\frac{\partial L}{\partial A} = [4, 22]$$

ReLU fonk geçir → Trev

$$\text{ReLU}(x) \quad x > 0 = [1, 1]$$

$$\frac{\partial L}{\partial z} = [4, 22] \quad \times \quad [4, 22] \\ \frac{\partial L}{\partial z} = [4, 22]$$

Ağırlıklar → Trev

ilk nöron

$$\begin{matrix} w_{11} = 1 \\ x_1 = 2 \end{matrix} \quad \left[z_1 = x_1 \cdot w_{11} + x_2 \cdot w_{21} + x_3 \cdot w_{31} \right]$$

$$1) \quad \frac{\partial z}{\partial w_{11}} = x_1 \rightarrow 2$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_{11}} = 8$$

$$2) \quad \begin{matrix} w_{21} = 2 \\ x_2 = 3 \end{matrix} \quad \frac{\partial z}{\partial w_{21}} = x_2 \rightarrow 3$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_{21}} = 12$$

$$3) \quad \begin{matrix} w_{31} = 3 \\ x_3 = 1 \end{matrix} \quad \frac{\partial z}{\partial w_{31}} = x_3 \rightarrow 1$$

$$\frac{\partial L}{\partial w_{31}} = \frac{\partial L}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{31}} = 4$$

$$[8, 12, 4]$$

! Aynı işlemleri ikinci nöron için de yap

$$\frac{\partial L}{\partial z_2} = 22 \text{ için yap}$$

$$[44, 66, 22]$$

İlk nöron $[8, 12, 4]$ (z_1)
İkinci nöron $[44, 66, 22]$ (z_2)

Ağırlık matrisine kay

13 / 09 / 2026

* Transpoz al

Ağırlık

Gradyent matrisi

$\frac{\partial L}{\partial w}$

$$\begin{bmatrix} 8 & 44 \\ 12 & 66 \\ 4 & 22 \end{bmatrix} \begin{matrix} x_1 \\ x_2 \\ x_3 \end{matrix}$$

Ağırlık güncellenen

0.1

$$W_{\text{ilk}} - \eta \cdot \frac{\partial L}{\partial w} = W_{\text{son}}$$

! Mayı ağırlığı ne kadar değiştireceğim?

Matrisi

$0.1 \cdot$

$$\begin{bmatrix} 8 & 44 \\ 12 & 66 \\ 4 & 22 \end{bmatrix} = \begin{bmatrix} 0.8 & 4.4 \\ 1.2 & 6.6 \\ 0.4 & 2.2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} - \begin{bmatrix} 0.8 & 4.4 \\ 1.2 & 6.6 \\ 0.4 & 2.2 \end{bmatrix} = \begin{bmatrix} 0.2 & -0.4 \\ 0.8 & -1.6 \\ 2.6 & 3.8 \end{bmatrix}$$

ilk ağırlık

Ağırlık değişimi

Yeni ağırlık matrisi

Loss = 125 puan

Bu değer sadece eğitimin doğru ilerleyip ilerlemediğini kontrol eden hata puanıdır

⊗

Aktar

- En az loss

- Gradyent (Trev) sıfır

mutlak minimum

Zincir Trevi

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial b}$$

$\frac{\partial L}{\partial b}$

Bias Gradyent

$$\frac{\partial L}{\partial z} = [4, 22]$$

$$z = x \cdot w + b$$

$$[4, 22] \cdot [1] = [4, 22]$$

$$b_{\text{yeni}} = b - \eta \cdot \frac{\partial L}{\partial b}$$

$$\frac{\partial z}{\partial b} = 1$$

$$b_{\text{yeni}} = [1, 2] - 0.1 \cdot [4, 22]$$

$$b_{\text{yeni}} = [0.6, -0.2]$$

BITTİ

2) input $\rightarrow X$ (girdi)

2) $Z \rightarrow (X_1 \cdot w_1) + (X_2 \cdot w_2) + b$ (Nokta Çarpımı) 13/08/2024

3) $A \rightarrow \text{Relu}(Z)$ (Aktivasyon Fonk sonucu)

output

Koda geçirme aşaması:

$$\frac{\partial L}{\partial A}$$

$$\frac{\partial L}{\partial Z} = \frac{\partial L}{\partial A} \cdot \frac{\partial A}{\partial Z}$$

$X \rightarrow Z \rightarrow A \rightarrow L$
Nokta Çarpımı Aktivasyon Fonk Loss değeri

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial A} \cdot \frac{\partial A}{\partial Z} \cdot \frac{\partial Z}{\partial X}$$

$F'(Z)$

$F'(Z)$

$$\frac{\partial L}{\partial w} = dZ \cdot W^T$$

Layer 1 $A^{(1)}$ çıktısını çetti output

Layer Sisteminde geçerse Layer 2 $A^{(1)}$ 'i input olarak alır

Layer 1

↓

Layer 2

↓

Output

! peki layer 2 alıyorsa $A^{(1)}$ kaybolacak mı?
Çünkü layer 1 ağırlıklarını nasıl değerlendireceğiz
 dA_{pre} kullanacağız

→ önceki layer'e ait

$dA \rightarrow A$ 'ya göre Loss gradienti $\frac{\partial L}{\partial A}$

Pre → önceki layer'a ait

! Forward içinde önceki ağırlıklar backwardda kullanılmak üzere saklanır

Backprop → ağırlıklar ne kadar değişmeli?

Update → ağırlıkları güncelle

ÖRNEK
DEVAM

Layer 1 → Layer 2 → output

$$dZ = [4, 22]$$

$$[92, 118, 144]$$

13/09/2026

$$W = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

$$\rightarrow W^T = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

gönder

$$dA_{pre} = dZ \cdot W^T$$

önceki layer için $\frac{\partial L}{\partial A_{pre}}$

$$\begin{bmatrix} 4 & 22 \end{bmatrix} * \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 92 & 118 & 144 \end{bmatrix}$$

Layer-2

x1 x2 x3 etkisi

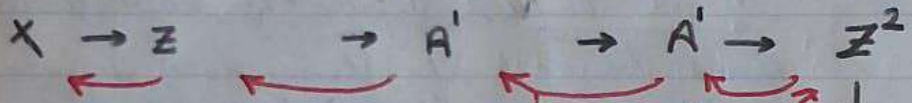
Layer 2 nin dZ'si → [4, 22]

Layer 1 'e gönder.

layer-1

layer-2

girdi → nokta çarpım → A₁ fonk → çıktı → nokta çarpım



$$A^2 \rightarrow Z^2 \rightarrow A'$$

$$dA'_{pre}$$

Loss a göre gradianti

$$dA_{pre} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

$$= \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

$$= \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

NOT

$$dW = \frac{\partial L}{\partial W}$$

ağırlıkların Loss fonk'a göre gradianti

Yeni görevimiz

$$[1, 2, 3, 4] \rightarrow$$

Tek input yerine

$$[1, 2, 3, 4]$$

$$[5, 6, 7, 8]$$

$$[9, 10, 11, 12]$$

Görel input

yapısına geçmek

Batch

14/09/2026

Şuan yaptığımız networkte tek örnek vermiştik

① $\text{input} = \text{np.array}([2.0, 1.0, 3.0])$
 $\text{target} = \text{np.array}([1.0, 1.0, 1.0])$

$x.\text{shape} = (3,)$

Batch birde fazla aynı anda örnek vermek:

$x = \text{np.array}([2.0, 1.0, 3.0],$
 $[1.0, 4.0, 2.0],$
 $[3.0, 2.0, 1.0])$
 $x.\text{shape} = (3, 3)$

OR

MNIST de

60k görüntü var

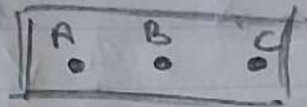
batch kullanılır

! kodun tek örneğe uygun halde çok örneğe
uygun hale getirilmesi

$x.\text{shape} = (3,)$

$w.\text{shape} = (3, 4)$

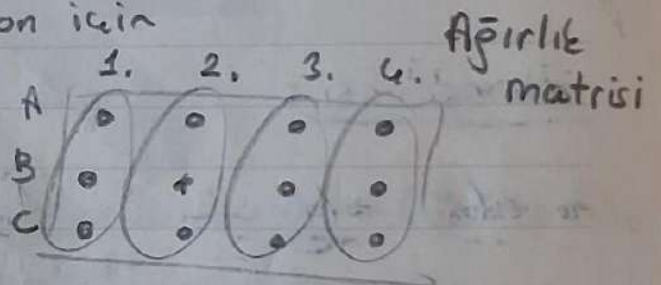
x



4 nöron için

$(4,)$ yarı
 $[z_1, z_2, z_3, z_4]$

4 output verir



batch moduna geç:

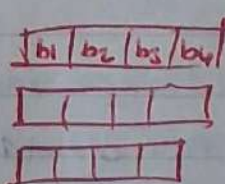
! bias aynı 4 bias bütün örneklere
uygulanır

$x.\text{shape} = (3, 3)$

$w.\text{shape} = (3, 4)$

x

$(3, 4)$ yarı



* Çünkü aynı
 $(4,)$ nörona
aynı bias
uygulanır

4 nöronun
output'u

$\begin{bmatrix} z_{11} & z_{12} & z_{13} & z_{14} \\ z_{21} & z_{22} & z_{23} & z_{24} \\ z_{31} & z_{32} & z_{33} & z_{34} \end{bmatrix}$

! Aktivasyon = $A.\text{shape} \Rightarrow (3, 4)$
fonk.

Batch $\rightarrow$ gradient ağırlığının ortalamasını al

14/09/2026

ÖRNEĞİN

Layer-1 de 1 nöron

$W = [w_1, w_2]$ ağırlıkları

batch de 3 örnek var:

$$X = \begin{bmatrix} 2 & 3 \\ 4 & 5 \\ 1 & 2 \end{bmatrix}$$

$$1. \text{örnek} \rightarrow [2, 3] \xrightarrow{dz} 10$$

$$2. \text{örnek} \rightarrow [4, 5] \xrightarrow{dz} 20$$

$$3. \text{örnek} \rightarrow [1, 2] \xrightarrow{dz} 30$$

$$\frac{dw_1}{dw} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w}$$

$$1. \text{örnek} \rightarrow x_1 = 2 \quad dz = 10$$

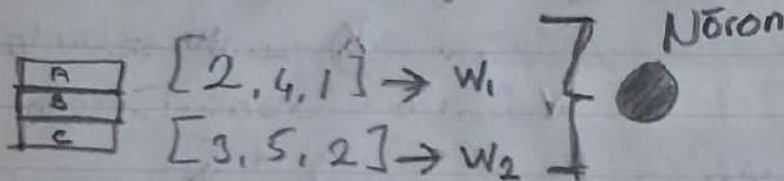
$$dw_1 = 20$$

$$2. \text{örnek} \rightarrow 4 \times 20 = 80$$

$$3. \text{örnek} \rightarrow 1 \times 30 = 30$$

aynı w_1 bütün bu 3 örnekte kullanıldığı için 3 örneğin etkisini birleştirir
 $20 + 80 + 30 = 130$

diğer nöron içinde yapılırsa $dw = [130, 190]$



db aşaması: $\frac{\partial z}{\partial b} = 1$ $\partial b = \partial z$

bias (db) $\rightarrow 60$

$$1. \text{örnek} [2, 3] \rightarrow dz \rightarrow 10$$

$$2. \text{örnek} [4, 5] \rightarrow dz \rightarrow 20$$

$$3. \text{örnek} [1, 2] \rightarrow dz \rightarrow 30$$

$$60$$

ÖRNEK

$$X = \begin{bmatrix} [2, 3] \\ [4, 5] \\ [1, 2] \end{bmatrix}$$

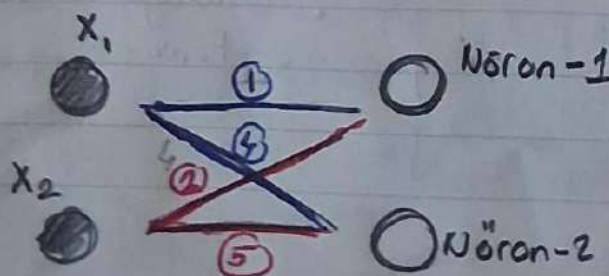
3 örnek var

her örnekte 2 input

layerde 2 nöron var

$$W = \begin{bmatrix} [1, 4] \\ [2, 5] \end{bmatrix}$$

batch işleri uygulanır toplamını al



Not

Örnek-1

input-1 input-2

2 3

Örnek-2

4

5

Örnek-3

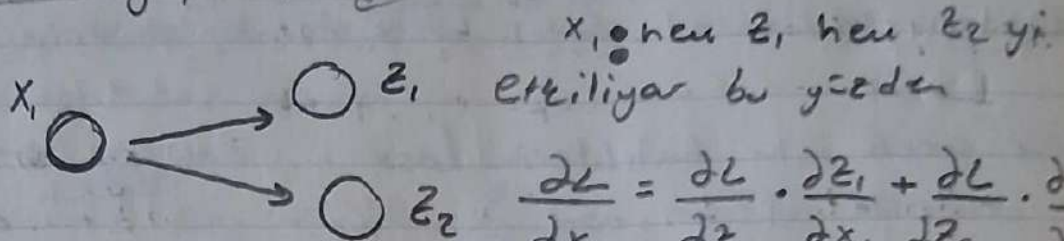
1

2

batch sayısı = 3 ✓

15 / 09 2026

* x_1 x d2 yap toplar



$$\frac{\partial L}{\partial x_1} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial z_1}{\partial x_1} + \frac{\partial L}{\partial z_2} \cdot \frac{\partial z_2}{\partial x_1}$$

w_{11} w_{12}

$\frac{\partial L}{\partial x_1} = dz \cdot w_{11} + dz \cdot w_{12}$

inputa giderken

yani $[dz \cdot w^T]$ $\rightarrow$ boyut için transpose al

* Forward: $x @ w$

* Backward: $dz @ w.T$

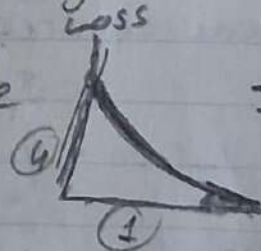
Batch olması matrisi değiştirmes. Sadece aynı işlemi 3 örneğe birde uyguluyoruz

$dw \Rightarrow$ Ağırlıkları nasıl değiştireceğin

$db \Rightarrow$ Biasları nasıl değiştireceğin

$4A-pre \Rightarrow$ önceki layera hangi gradianti göndereceğin

$\frac{\partial L}{\partial w} = -4$ gibi negatif değerde

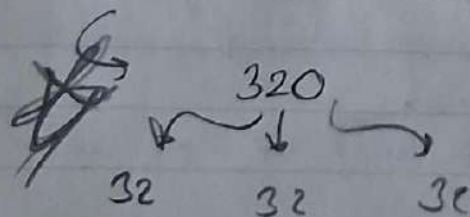


$\frac{-4}{1} = -4$ eğimi $\downarrow$ loss

$\rightarrow$ epoch-1 $\rightarrow$ 1 update

Full-batch $\rightarrow$ bütün dataseti eğitime alır

Mini-batch $\rightarrow$ dataseti parçalara ayırır



10 tane paket şeklinde eğitime alınır

Mini-batch:

Epoch 1

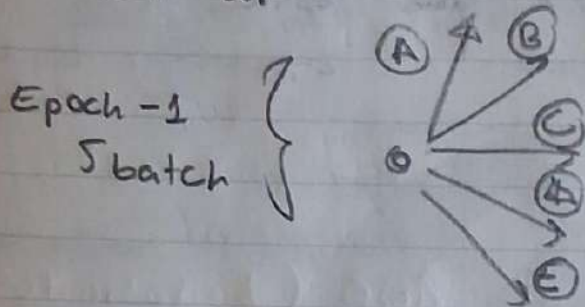
16/09/2026

batch 1 → forward → backward → update
batch 2
batch 3

epoch → dataseti batch böl → her batch için train-step

her epoch için batchlerin loss ort al

sonraki epoch ✓



$$\frac{A+B+C+D+E}{5} = \text{ort loss}$$

* hataları toplamak ✗

* hataların ortalamasını almak ✓

8 örnek 3 örnekten daha çok hata çıkartır toplamak (sum()) yerine mean() kullan ortalamayı al ✓

* mean() → burada da ort alırken faktörü kullan
Loss → mean squared Error

$$L = \frac{1}{N} \sum (y_{\text{true}} - y_{\text{pred}})^2$$

$$\frac{\partial L}{\partial y_{\text{pred}}} = \frac{1}{N} \cdot -2(y_{\text{true}} - y_{\text{pred}})$$

↳ y_true.size [3, 2] için → 6 gibi

Sorun

10 input

batch 3

batch 3

batch 3

batch 1

bunun ağırlığı
daha * olsun ki
değeri paylaştırılır
olsun

16/09/2026

Bunun çözümü: $loss * batch_num$ yani 1 tane data için
1 ile karpıp total loss elde

* $Loss += loss * batch_num$

<u>loss</u>		<u>batch</u>		
2	*	2	=	4
5	*	5	=	25
5	*	1	=	5

} yani toplam 34 loss
puanı olacak

başlıyoruz

Shuffle

Shuffle olmadan:

Epoch 1:

[1, 2, 3] → [4, 5, 6] → [7, 8, 9] → [10]

Epoch 2:

[1, 2, 3] → [4, 5, 6] → [7, 8, 9] → [10]

Epoch 3:

[1, 2, 3] → [4, 5, 6] → [7, 8, 9] → [10]

dataset aynı sıra
ile işleme alınır
aynı ağırlıkları vere-
rek aynı sonuç
bulunur.

Shuffle ile:

Epoch 1:

[7, 2, 9] → [1, 5, 3] → [8, 4, 6] → [10]

Epoch 2:

[4, 1, 8] → [6, 10, 2] → [5, 9, 3] → [7]

[3, 2] [5, 6]

[1, 4] → [3, 2]

[5, 6] [1, 4]

* $index = [2, 0, 1]$

$index = np.random.permutation(len(input))$

↳ bu belli bir index belirtir [3, 0, 4, 1, 2] gibi

→ input verisi uzunluğu kadar rastgele indeks 17/08/2026
 $indices = np.random.permutation(len(input))$

$len(input) = 4$ $indices = [2, 0, 3, 1]$

Optimizer

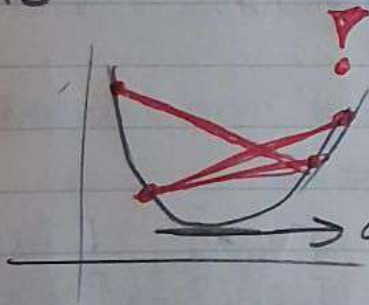
problem → learning-rate ; bazı çok küçük ve çok büyük olabilir.

örneğin eğitimi de sıklıkla yaklaştıkça learning rate ↓ olmalıdır artık ciddi ağırlık değişimine ihtiyacımız ✗



learning-rate = 0.0 çok küçük ↓ = hata miktarı belli ama bu hatanın çok küçük bir oranını düzeltiyoruz

* Optimizer ⇒ gradient kullanarak ağırlıkları nasıl daha akıllıca düzelebiliriz? cevap, → kontrollü learning rate



! büyük learning-rate ile min noktası daha zor olur/hatta imkânsız

→ ancak min (ekstremin) noktasını bulmak

* Momentum → sadece şu anki gradiyente ✗
önceki gradiyentlere de bak. önceki adımların yön ve büyüklüğü ile yeni adımı tayin etmeye yarar

! momentum ağırlık değişimlerine yansıtmaya yarar

velocity ve beta tanımla

hız önceki hızın ne kadarını koruyacak

17/09/2016

$$\begin{cases} \text{weight} - = \text{learning-rate} * \text{gradient} \\ \text{velocity} = \text{beta} * \text{velocity} - \text{learning-rate} * \text{gradient} \\ \text{weight} + = \text{velocity} \end{cases}$$

Başlangıç $\text{velocity} = 0$
 $\text{learning-rate} = 0.1$
 $\text{beta} = 0.9$ (0.80 eski hızı korur)

④ $\text{gradient} = 5$ ise,
$$v = 0.9 \times 0 - 0.1 \times 5$$
$$= -0.5$$

$$\text{weight} + = -0.5$$

2. tur

$$\text{velocity} = -0.5$$

$$\text{learning-rate} = 0.1$$

$$\text{beta} = 0.9$$

$$v = 0.9 \times -0.5 - 0.1 \times 5 = -0.95$$
$$\text{weight} + = -0.95$$

YANI: -0.5

YERİNE

-0.5

* Şeklinde azalma yönünde daha net hızlı adımlar atmanız, sağlanır.

-0.5



-0.95

-0.5

-1.355

-0.5

...

-0.5

* Ayrıca yumuşatma yönünde:

-0.5

YERİNE

-0.5

* Şeklinde belirtilen yönde daha yumuşak (soft) iletmeyi sağlar. Ani değişimlerde kaçınır.

-0.5



-0.95

0

-0.82

0.5

-0.21

0.5

0.32

* Aynı yöndeki gradientleri daha hızlandırarak sağlar

* Ani değişimde model daha az etkiler önceki adımların ağırlığı ve yönü korur.

! Gradientlerin bir miktar geciktirileceği olduğu durumlarda faydalıdır.

① layer → gradient üretir

→ optimizer → gradienti kullanarak
update() yapar

17/09/2026

~~diğer sayfa~~
vein vadede :

layers.py

optimizer.py

Loss-functions.py

} formatına geçmek lazım

* momentumun amacı : geçmiş gradientlerin hareket bilgisini biriktirerek güncellemeleri daha kararlı ve baz yerlerde daha hızlı hale getirmektir.

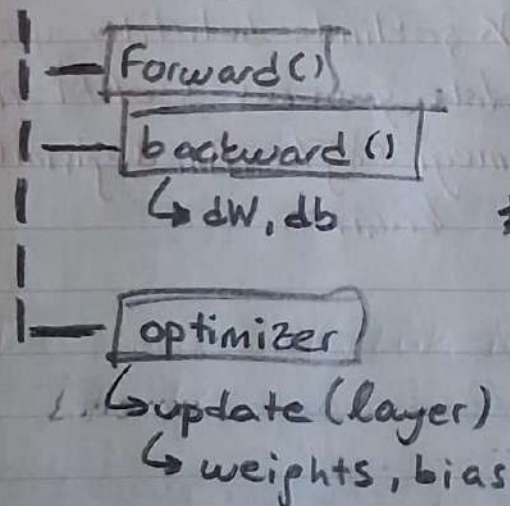
* gradient → velocity-1 → velocity-2 ... hız sarsma büyüklüğüne dikkatli diye (B) ile %sel olarak ↓

optimizer = 1) SGD [Stochastic Gradient Descent]
(öngörülemez)

learning-rate

↳ Neural Network → optimizerin sorumluluğunda
bunun içinde kullanacak

Neural Network



Stateless

* Normal SGD = Her adımda sadece mevcut gradiente bakar

Statefull

* Momentum ise geçmiş hız korur

* Velocity matrisinde ilk seterde $\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ matrisi oluştur
daha sonra yeni hız değerlerini bu matrise ekle

SGD : gradient $\rightarrow$ update

momentum : gradient + velocity $\rightarrow$ update

17/09/2026

momentum $\Rightarrow$ gradient hangi yönde gidiyor

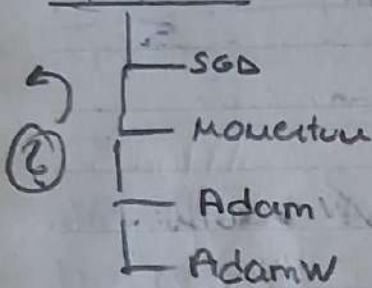
velocity $\Rightarrow$ gradient ne kadar büyük / değişken

Optimizer 2) ADAM

NOT

SGD ve ADAM gibi momentum da optimizierdir.

optimizer



*** Farklı gradienti kullanarak weighti nasıl güncelledikleri

Ben momentum ve SGD iç içe sandım farklı optimizierlermiş

! SGD $\rightarrow$ sadece o anki gradient var

$$W = \text{learning-rate} * \Delta W$$

! momentum $\rightarrow$ önceki gradientleri de kullanır

velocity (ve Beta) değişkenleri tutar

$$v_t = \beta v_{t-1} - \eta \nabla W_t$$

$$W_t = W_{t-1} + v_t$$

Optimizer - 3) Adam [Adaptive moment Estimation]

1. moment

gradientlerin hareketli ortalaması

(genel hareket yönü)

2. moment

gradientlerin karelerinin hareketli ortalaması

(gradient büyüklüğü)

Adam:

17/08/2026

update

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\theta_{t+1} = \theta_t + \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

SGD genel problemi: Sadece o anki gradiente bakar

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Simdiki gradient

geçmiş + şuan

önceki gradient

önceki gradient hafızası

ör $\beta = 0.90$ için

ÖRNEK

önceki $m = -4$

0.90 önceki gradient

0.10 yeni gradient

kullanılan gradient

şimdiki gradient = -6

$\beta = 0.90$

$(1 - 0.90)$

$$(0.90 * -4) + (0.10 * -6) = -4.20$$

ikinci hafıza:

neden gradientin karesi?

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

-5	25
+5	25
-5	25
+5	25
	100

Yön önemli & sadece büyüklük için kullanılır

ÖRNEK	NORMAL	Adam
-5	→	25
+5	→	25
-5	→	25
+5	→	25
		100/4
		= 25 ✓

geçmiş + şuan

ÖRNEK-1

Önceki $r=4$
 gradient = -6
 $B_2 = 0.9$

$$0.90 * 4 + \frac{(1-0.90) (-6)^2}{2}$$

17/09/2026

$\gamma = 7.2$ gibi büyüklük elde ederiz

$\gamma \Rightarrow$ geçmiş gradient bilgilerimizi birleştiriyoruz hafıza

1. Adım gradient $\rightarrow p_1 \rightarrow \gamma_1$

2. Adım gradient $\rightarrow p_2 \rightarrow \gamma_2$

* gradient her adımda hesaplanır
 * V her adımda yeniden üretilir.
 $\gamma_t = B_2 \gamma_{t-1} + (1-B_2) p_t^2$

ÖRNEK-2

$$\gamma_0 = 0$$

$$p_1 = 5$$

$$0.90 * 0 + 0.10 * 5^2$$

$$\gamma_1 = 2.5$$

$$p_2 = 3$$

$$0.90 * 2.5 + 0.10 * 3^2$$

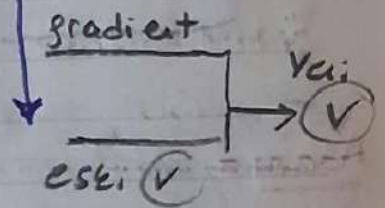
$$\gamma_2 = 3.15$$

$$p_3 = 7$$

$$0.90 * 3.15 + 0.10 * 7^2$$

$$\gamma_3 = 7.735$$

* V (büyüklük) hafızası bu şekilde oluşturulur



Adım:

$$\frac{m_t}{\sqrt{\gamma_t} - \epsilon}$$

yön

γ kavre tuttuğu için kök al $\rightarrow \sqrt{\gamma_t}$

büyüklük

payda 0 olmasın diye 10^{-8} gibi küçük değer

* γ yön $\rightarrow$ büyüklüğe bölme nedeni = direkt son gradienti belirtilen yönde ilerletme eski büyüklüğe böl ki adım büyüklüğümüz ne olmalı anlayalım

$$m_+ \rightarrow 100$$

$$\frac{100}{\sqrt{2500}} = 2$$

$$50\% \quad v_+ \rightarrow 2500$$

$$\sqrt{2500}$$

Adım büyüklüğüne böl

Çünkü normal mi bu gradient alır!

$$m_+ \rightarrow 200$$

$$\frac{200}{\sqrt{10000}} = 2$$

$$100\% \quad v_+ \rightarrow 10000$$

$$\sqrt{10000}$$

gradientin süreci

$\hat{m} \rightarrow$ yönünü m da alır

$\sqrt{F + \epsilon} \rightarrow$ adım büyüklüğünü sağlamış olur

17/09/2026

ÖRNEK

(A) $\boxed{g = 0.1}$ $\epsilon = 0.01$ gradient
 $v = 0.01$
 $m = 0.08$

(B) $\boxed{g = 10}$ büyük gradient
 $v = 100$
 $m = 8$

! Kedi

ölçeğine

göre

değerlerdir

$$\frac{0.08}{\sqrt{0.01}} = 0.8$$

Aynı çıktı

$$\frac{8}{\sqrt{100}} = 0.8$$

! her ikisinin de adımı orantılı olarak gradient değişimi aynı çıktı, yani $\uparrow$ ve $\downarrow$ gradientte belli v ve m değerleri aynı çıkıyor

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{m_t}{\sqrt{v_t + \epsilon}}$$

learning-rate bu
sayede efektif
davrane

$\eta \times 100 = ?$ uygun olan
 $\eta \times 5 = ?$ koşulda
uygun learning
rate kullanılır olur

Adam =

Gradianti, 0

weight' in peşin

ölçeğine göre normalize eder.

$$w_1 \text{ gradient} \rightarrow [0.01, 0.02, 0.015, 0.01]$$

$$w_2 \text{ gradient} \rightarrow [10, 12, 8, 11]$$

Büyüklik $\emptyset$

$m = 10$ itisi
içinde normal
olmalı,
ANA $\emptyset$

Problem

$$\begin{cases} m_0 = 0 \\ y_0 = 0 \end{cases} \left\{ \begin{array}{l} \text{icin gradient} = 5 \text{ olsa bile} \\ \text{ilk deger:} \end{array} \right.$$

$$0.90 \times 0 + 0.10 \times 5$$

Başlangıç gradianti

! Gekiyor

$$\boxed{m = 0.5}$$

Bias correction gözetim

17/09/2026

Adam optimizatoru diyor ki:

"Ben ilk değeri gereğinden $\downarrow$ gördüm bunu düzeltmem lazım"

Sonuçunda: $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ ve $\hat{y}_t = \frac{y_t}{1 - \beta_2^t}$ kullanılır

örnek $t=1$ için

$\frac{0.5}{1 - 0.9} = 5$! ilk adımda bu yöntemi kullanılır

ÖRNEK

weight = 2
m = 0
y = 0
gradient = 5

$\beta_1 = 0.9$
 $\beta_2 = 0.999$
 $\epsilon = 1e-8$

$\alpha = 0.1$ (1-m ve y)

$m = (0.9 * 0) + (0.1 * 5)$

$m = 0.5$

$y = (0.999 * 0) + (0.001 * 25)$

$y = 0.025$

2-bias correction)

$m = \frac{0.5}{1 - 0.9} = 5$ ✓ ilk değer

3-learning-rate)

0.1

1

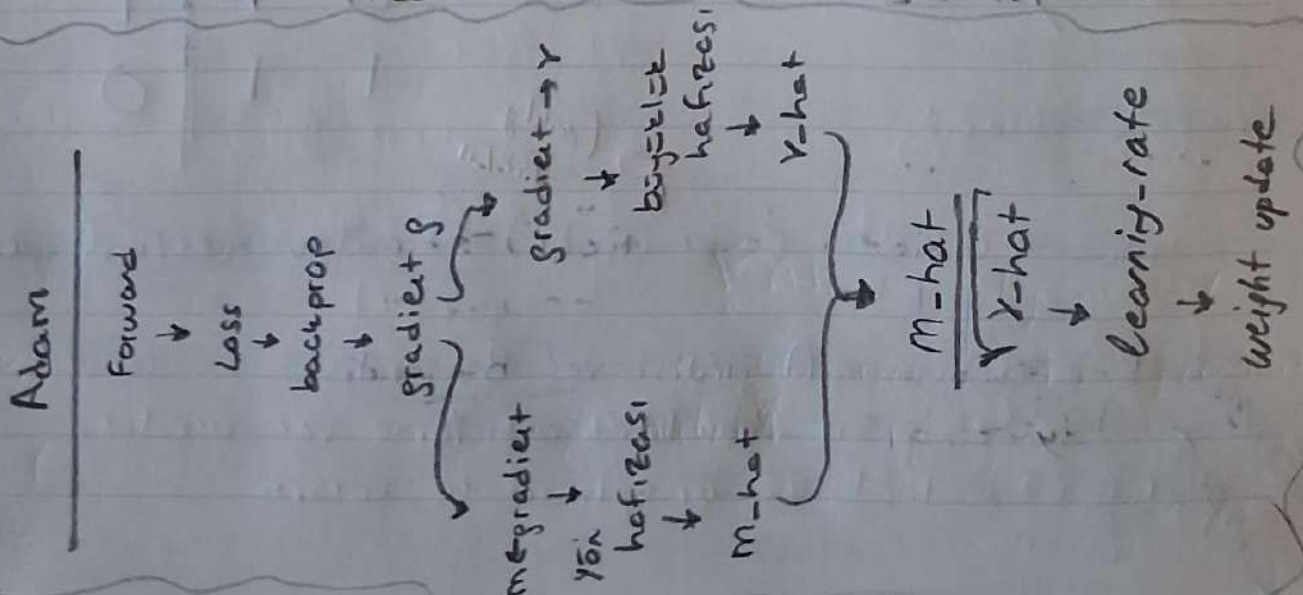
$\Delta W = \alpha \cdot W$

$y_{yeni} = \frac{0.025}{1 - 0.999} = 25$ ✓

$\Delta W_{yeni} = \Delta W - 0.1 * 1$

$W = \frac{m}{\sqrt{y}} \rightarrow \frac{5}{\sqrt{25}} = 1$

$\Delta W_{yeni} = 1.9$



Ağırlık matrisi

18/09/2026

$$dw = \begin{bmatrix} 2, 1, 3 \\ 4, 7, 6 \\ 5, 9, 8 \end{bmatrix} \quad dw[0,0] = 2$$

her ağırlığın kendi m ve v değeri olmalıdır

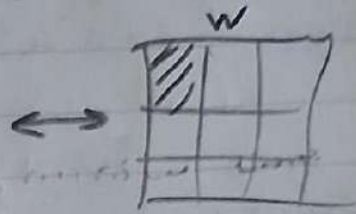
Yani $m[0,0]$ ile $v[0,0]$ gibi

hafıza her ağırlık için ayrı ayrı tutulur

Ağırlık

$mw \rightarrow w$ ile aynı boyut

$vw \rightarrow w$ ile aynı boyut



bias

$mb \rightarrow b$ ile aynı boyut

$vb \rightarrow b$ ile aynı boyut

Adım = $dw \rightarrow mw/vw$ güncelle $\rightarrow$ bias correction $\rightarrow$ normalize

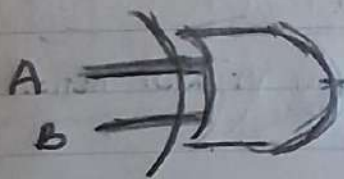
her update sonucu m ve v kaybolmamalıdır.

w güncelle

$dw \rightarrow$ ağırlık gradıent

$mw \rightarrow$ hafızadaki gradıent

XOR Gate Project



$$A'B + AB'$$

A	B	output
0	0	0
1	0	1
0	1	1
1	1	0

Projenin amacı:

Leaky ReLU

input $\rightarrow$ 4 hidden layer (ReLU) $\rightarrow$ 1 output neuron (sigmoid)

Görnek
2 input

$\begin{cases} 00 \\ 10 \\ 01 \\ 11 \end{cases}$

A girdisi ve B girdisi

* Test sonuç

matris boyutları

19/08/2026

$x = 2, 4 \rightarrow$ A ve B inputu
 $\downarrow$

Layer-1 = 4, 1 $\rightarrow$ 1 adet sonuç
 $\downarrow$

Output = 4, 1 ∇ batch-size = 4 yapılarak 4 örneğe bölünür oluruz

∇ Relu kullanma Hatası ∇

Relu belli bir epochtan sonra negatifleri sıfırlar ve türevi olmadı için sadece tek değer kalır

$[---, ---, ---] \rightarrow [---, 0, 0]$

Sonuçta ta ortalama aldığımız için $---/3$ olur

∇ Gözünüzde leakly-relu kullanmak

Relu sonucunda: $[0.006...]$ beklenen = $[0, 1, 1, 0]$
 $0.666... \quad 0.666... \quad 0.666... \quad 0.666...$

hidden-layer-output $0.666...]$

$[1.517...]$ $[0, 1] \rightarrow 1$
 $[0, 0, 0, 0]$ $[1, 0] \rightarrow 1$ $\frac{1+1+0}{3} = 0.666...$
 $[0, 0, 0, 0]$ $[1, 1] \rightarrow 0$
 $[0, 0, 0, 0]$ Sistem bu ortalamasını

3 nöronu birbirinde ezberledi
ayıramadığı için ort aldı.

Leakly-Relu Sonucu

500 epoch ile: $[0, 0] \rightarrow 0.0073$ $\checkmark$ Test başarılı
 $[0, 1] \rightarrow 0.9793$
 $[1, 0] \rightarrow 0.9792$ model öğrendi
 $[1, 1] \rightarrow 0.0252$

19/09/2026

Relu :

$$z < 0$$



$$\text{output} = 0$$



$$\text{gradiant} = 0$$

* Nöron öğrenmeyecebilir

Leaky-Relu

$$z < 0$$



$$\text{output} = z$$



$$\text{gradiant} = 1$$

* gradiant tamamen kaybolmaz

Analitik Gradiant

$$L(w) = (wx - y)^2$$

$$\frac{dL}{dw} = 2(wx - y) \cdot x$$

→ ikisini de backward() için kontrol et

eğer fark ↓↓↓
backward() değeri
çalışıyor

Nümerik Gradiant

w'yi az değiştirirsen loss ne olur?

$$\frac{dL}{dw} = \frac{L(w+h) - L(w)}{h}$$

$$h = 1e-5$$

||| ————— ||| orta noktayı bul

$$\frac{L(w+h) - L(w)}{2h}$$

Predict() fonksiyonu ile kullanıcı modele tahmin yap der
→ Bonus için eğer uygun ağırlıklar history de saklanabilir lazım

Layer

↳ Forward()

↳ Backward()

↳ Gradiant üret

↳ ~~update()~~

optimizer

↳ gradiant kullan

↳ weight / bias güncelle

↳ update()

→ bunu sil Neural Network içine ver orada ki
update() methodu da işi optimize et versin

13/09/2024

mimari [sinir ağı işlemleri] Neural Network

- └ Forward() → veri atışı
- └ predict() → sadece tahmin
- └ backward() → gradient geri taşı
- └ update() → optimizere sorumluluk taşır
- └ train_step() → bir eğitim adımı
- └ Fit() → tüm eğitimi yönetir

[Tek nöron işlemleri] Neuron

- └ Forward()
 - └ gradient()
 - └ update()
- estü tek nöronlu eğitim uyarısı

Optimizer

- └ update(layer) → güncellemeleri yapar

[katman işlemleri] Layer

- └ Forward() → ileri besleme hesaplar
- └ Backward() → geri besleme hesaplar

Neural Network, forward() } tek görevleri asıl mat
backward() } işlemleri olan
layer.forward() { çağırarak
layer.backward() }

Not

Bir fonk içinde self ile tanımlı attribute varsa return

ile dışarı çıkartırsan diğer fonkta erişebilirsin

* Class Layer:

```
def backward():
```

```
    self.dw = None
```

```
    self.db = None
```

```
def ...():
```

```
    dw, db = layer.backward()
```

create

Layer.backward

└ dw

└ db

└ dA_pre (Return)

kullan-1

NeuralNetwork.backward()

└ dA_pre önceti layer için

kullan-2

Optimizer

└ dw

└ db

güncellenmede kullan